Save Point

Quiz App AI (Flutter)

AlotaNotes Quiz App · Prepared for Logan Young, Save Point Limited · 23 October 2026

A mobile-first notes + spaced-repetition quiz app. Keep notes organized into subject folders, generate quiz questions from them using your own connected AI account, and get reviewed on a schedule that stretches out the better you know something.

This is a from-scratch Flutter rewrite of an earlier Python/Tkinter desktop app (`Quiz-app-Ai`, a sibling project): it reuses that project's *ideas* (and, where useful, its algorithms ported to Dart), not its code. See `Docs/architecture.md` for how the two relate.

- **Local-only for now.** No backend, no real accounts. Sign-in is a single hardcoded local session (`admin` / `1234`) behind a swappable interface, meant to be replaced by a real auth provider later without touching any screen (see `Docs/future_work.md`).
- **Bring your own AI key.** Connect OpenAI (or any OpenAI-compatible endpoint: OpenRouter, Groq, a local Ollama/LM Studio server) or Anthropic directly from the Account tab. The app itself is never billed.
- **Import your own files.** `.txt`, `.md`, `.pdf`, `.docx`, `.pptx`, single file or bulk, with a preview/reorder step before anything is actually imported.
- **Spaced repetition**, a fixed interval sequence (`1, 2, 3, 5, 7, 14, 30` days), looping back to the start once it completes; any wrong answer resets it. See `Docs/scheduling_algorithm.md`.

Quick start

Requires the Flutter SDK (this project was built against 3.47.2) and either a connected device, a running emulator, or `flutter build apk` for a standalone install.

```
flutter pub get
flutter run
```

See `Docs/running_locally.md` for full setup, first-run notes, and where data lives on-device.

Testing

```
flutter analyze
flutter test
```

175 tests covering pure logic (scheduling, grading, text extraction), storage repositories (real in-memory SQLite), AI providers (HTTP-mocked, no real network/API key needed), and widget tests for every screen (real `tester.tap()` interaction, not device input simulation). Line coverage of `lib/` is 100%; `flutter test --coverage && dart run tool/check_coverage.dart` checks it. See `Docs/testing.md` for why the device-input distinction matters: it's the difference between a test that's actually reliable and one that isn't, and this project hit that difference the hard way once already.

There's also one end-to-end test that runs the real app on Linux desktop (real SQLite file, real settings and keychain, real HTTP to a local fake AI server), from sign-in through quiz, import and export:

```
flutter test integration_test -d linux
```

It needs the Linux desktop toolchain installed; see `Docs/testing.md`.

Documentation

Everything needed to understand and extend this codebase lives in `Docs/`:

- `architecture.md`, module map, layering, how this relates to the original Python desktop app
- `database_schema.md`, the SQLite schema
- `scheduling_algorithm.md`, the spaced-repetition interval algorithm
- `ai_providers.md`, the AI provider abstraction (OpenAI/Anthropic), how to add a third
- `gui_theming.md`, the visual design system
- `testing.md`, how to run tests and check coverage, the test fakes and patterns, what's not covered, and the ADB-tap cautionary tale behind why widget tests are the trustworthy path here
- `running_locally.md`, setup, first run, where data lives, known Android build gotchas
- `packaging.md`, the Windows installer / Linux AppImage release build, triggered automatically on a GitHub release
- `hosting.md`, the web build in Docker, and putting it behind a reverse proxy at alotanotes.savepoint.co.nz
- `pdf/`, the docs above as Save Point branded PDFs, rebuilt with `dart run tool/build_docs.dart`
- `future_work.md`, known, deliberate deferrals: what's blocked on things only a human can provide (developer accounts, real devices, brand assets), and what's just not built yet